

CCMS- Campus Complaint Management System



Shripad Waikar (2201188)

Advisor: Dr. Rohit Triphati Sir

Department of Computer Science and
Engineering Indian Institute of Information
Technology Guwahati

This report is submitted for the course of
CS410 : Project-I

Contents

1 Introduction

2 App Purpose

3 Target Audience

4 Core Features

5 Technologies Used

6 System Architecture

7 Security Implementation

8 Future Scope

9 Conclusion

10 Screenshots

11 Appendix

1. INTRODUCTION

Overview

Campus Complaint Management System (CCMS) is a full-stack web-based application designed to streamline complaint handling in academic institutions. Students can submit complaints, track status, and receive timely updates, while staff and administrators manage complaints centrally and consistently.

Problem Statement

Traditional complaint handling in universities is often manual, slow, and opaque. Complaints suffer from delayed response, missing prioritization, no enforcement of deadlines, and weak accountability, which reduces satisfaction and trust.

Solution

CCMS solves these problems with AI-powered classification using Google Gemini, role-based access control for Student, Staff, and Admin, real-time WebSocket notifications, automated SLA tracking with escalation, and PDF reporting. The frontend uses React 18, TypeScript, and Vite; the backend uses FastAPI, PostgreSQL, and SQLAlchemy async; authentication uses JWT; email uses Resend API.

2 APP PURPOSE

The primary goal of CCMS is to digitize the complaint workflow in universities, improve transparency, and ensure faster, measurable resolution.

- Digital complaint submission: Enables students to submit complaints online with structured details
- AI-based classification: Categorizes complaints into standard domains and priority levels
- Automated routing: Directs complaints to the correct department automatically
- SLA enforcement: Applies deadlines and escalates breaches automatically
- Real-time notifications: Sends instant updates through WebSocket and email
- Administrative analytics: Provides dashboards and reports for decision making
- In-app communication: Lets students and staff exchange clarifications inside the platform
- Feedback collection: Captures satisfaction via a 1–5 star rating system

3 TARGET AUDIENCE

CCMS serves all stakeholders in a campus complaint ecosystem by improving communication and operational control.

- Students: Submit issues, monitor progress, and receive updates
- Staff members: Handle assigned complaints within their department
- Administrators: Manage users, departments, SLAs, and reports
- University management: Review performance metrics and trends
- IT administrators: Configure, maintain, and secure the system

4 CORE FEATURES

4.1 AI-POWERED COMPLAINT CLASSIFICATION

The system uses AI to classify complaints accurately and speed up routing.

4.1.1 Google Gemini Integration

- gemini-1.5-flash model via google-generativeai SDK
- Structured JSON output: category, priority, suggested_department, reasoning, confidence_score
- Strict prompt control with fixed categories and priorities
- Async timeout: 8 seconds with fallback classification
- Validation layer replaces invalid outputs with safe defaults
- Confidence scoring stored per complaint for analytics
- AI acceptance tracking measures category accuracy

4.1.2 Auto-Department Routing

- AI suggests a department name
- Backend fuzzy-matches against existing departments

- Automatic assignment on submission
- SLA deadline set based on matched department policy

4.2 COMPLAINT MANAGEMENT

This module covers complaint submission, lifecycle transitions, and attachments.

4.2.1 Complaint Submission (Student)

- Complaint form with title, description, optional attachment
- Unique reference number format: CCMS-YYYYMMDD-XXXX
- AI classification runs immediately on submission
- SLA deadline auto-calculated by priority and department
- Attachment URL stored and served from /uploads/
- Initial audit trail entry created on submission

4.2.2 Complaint Lifecycle & Status Transitions

- Status flow: SUBMITTED → ASSIGNED → IN_PROGRESS → PENDING_INFO → RESOLVED → CLOSED / ESCALATED
- Validated transitions prevent invalid state changes
- SLA pauses during PENDING_INFO and resumes afterward
- Status updates logged in complaint_updates with author and timestamps
- RESOLVED prompts the student for rating
- Rating submission transitions complaint to CLOSED

4.2.3 File Attachments

- Students attach files during submission
- Staff attach files during updates
- Files served via FastAPI StaticFiles at /uploads/
- Max upload size is 10MB

4.3 SLA MANAGEMENT

SLA management ensures timely complaint resolution with automated monitoring.

4.3.1 SLA Policies

- Default SLA hours: CRITICAL=4, HIGH=24, MEDIUM=72, LOW=168
- Department-specific SLA overrides available
- Unique constraint for each department and priority pair
- Warning threshold at 80% elapsed time

4.3.2 Automated SLA Enforcement

- APScheduler job runs every 15 minutes
- Tracks active complaints in SUBMITTED, ASSIGNED, IN_PROGRESS
- Skips complaints with paused SLA in PENDING_INFO
- Sends SLA_WARNING at 80% elapsed time
- Auto-escalates to ESCALATED on breach
- Bumps priority by one level on breach

4.3.3 SLA Countdown UI

- Live countdown display on complaint pages
- Color-coded urgency: green, yellow, red
- Shows time remaining in days, hours, and minutes

4.4 REAL-TIME NOTIFICATIONS

Real-time updates keep users informed instantly.

4.4.1 WebSocket System

- Endpoint at /ws?token=<JWT>
- Token validated on connect; invalid tokens rejected
- Supports multiple tabs per user
- Ping/pong heartbeat keeps connections alive
- Broadcasts events such as submission, assignment, status changes, SLA warnings, messages, and resolution
- Notifications persisted in database for later retrieval

4.4.2 Notification Bell

- Unread count badge displayed on bell icon
- Notification history accessible on click
- Mark-as-read feature available
- Component subscribes to WebSocket on mount

4.5 MESSAGING SYSTEM

In-app messaging supports complaint-centric collaboration.

4.5.1 In-App Chat

- Complaint-specific chat channel between student and staff
- Messages stored with sender, complaint reference, and timestamp
- New messages trigger WebSocket notifications
- History loaded per complaint detail page

4.6 USER & ROLE MANAGEMENT

Role-based access defines user capabilities across the system.

4.6.1 Three-Role System

- Student: Submit complaints, view own complaints, chat, rate resolved complaints
- Staff: View assigned complaints, update status, attach files, chat
- Admin: Manage users, departments, SLA policies, analytics, and reports
- Role enforcement via JWT claims and FastAPI dependency injection

4.6.2 User Management (Admin)

- Create users with any role
- Assign staff to departments
- Activate or deactivate accounts (soft delete)
- Approve or deny pending staff registrations
- Trigger password reset emails via Resend API
- Search and filter users by role and status

4.7 DEPARTMENT MANAGEMENT

Department management organizes complaint routing and policy configuration.

- Admin can create, view, and manage departments
- Departments have names and staff members
- Complaints routed based on AI-suggested department
- Per-department SLA policies configurable by priority
- Performance tracked with average resolution time and escalation counts

4.8 ANALYTICS DASHBOARD (ADMIN)

Analytics provide visibility into complaint operations.

4.8.1 Live Analytics

- Total complaints and monthly trends
- Status distribution across workflows
- Category and priority breakdown
- Average resolution time
- SLA compliance rate calculation
- AI acceptance rate metric
- Department performance table
- Staff leaderboard with ratings and resolution time

4.8.2 PDF Report Generation

- Admin selects date range for reports
- ReportLab generates styled A4 PDF
- Includes cover page, status table, category table, averages, and leaderboards

4.9 AUTHENTICATION SYSTEM

Authentication secures access and session handling.

4.9.1 JWT Authentication

- HS256 access and refresh tokens
- Token type validation to prevent misuse
- Blacklist invalidates logout tokens
- bcrypt password hashing via passlib
- Password reset link with 1-hour expiry

4.9.2 Session Management

- Frontend stores token in Zustand and localStorage
- Axios interceptors attach Authorization headers
- Automatic refresh on 401 responses
- Logout invalidates token server-side

4.10 RATING SYSTEM

Ratings capture user satisfaction after complaint resolution.

- Students rate resolved complaints 1–5 stars
- Optional text feedback supported
- Rating linked to complaint and assigned staff
- Database CHECK constraint enforces score range
- Unique rating per complaint
- Rating submission closes the complaint
- Staff average rating shown in admin analytics

5 TECHNOLOGIES USED

Category | Technology

Frontend | React 18.3, TypeScript 5.4, Vite 5.3, Tailwind CSS 3.4

UI Components | Lucide React

Backend | FastAPI 0.111, Python 3.11, Uvicorn 0.30

ORM & Database | PostgreSQL 15+

Authentication | python-jose, passlib[bcrypt]

AI Integration | Google Gemini API, google-generativeai 0.7.2

File Uploads | python-multipart, aiofiles, FastAPI StaticFiles

Real-Time | FastAPI WebSocket, custom WebSocketManager

Development Tools | Git, GitHub, VS Code, Postman, pgAdmin

6 SYSTEM ARCHITECTURE

CCMS follows a modern two-tier architecture linking the web frontend, backend API, and database with external AI and email services.

1. Web Frontend (React + Vite)

- SPA served in development at localhost:5173
- Client-side routing with React Router v6
- Role-based protected routes via ProtectedRoute
- React Query for server caching
- Axios interceptors for JWT handling /
- WebSocket connection per authenticated session
- Pages organized by /student, /staff, /admin

2. Backend API (FastAPI + PostgreSQL)

- RESTful API at /api/v1/
- Routers for auth, complaints, messages, notifications, admin, departments
- Async SQLAlchemy sessions with asyncpg
- APScheduler SLA enforcement job every 15 minutes
- Static file serving for uploads at /uploads/
- WebSocket endpoint at /ws

3. Database (PostgreSQL)

- Tables: users, departments, complaints, complaint_updates, messages, notifications, ratings, sla_policies, token_blacklist
- UUID primary keys and enum types for roles, statuses, priorities, update types
- Unique constraints on SLA policy pairs and complaint ratings
- Check constraint for rating scores

4. External Services

- Google Gemini API for AI classification
- Resend API for email verification, password reset, and alerts

Architecture Diagram:

Browser ↔ React Frontend ↔ FastAPI Backend ↔ PostgreSQL
 ↘ Google Gemini API
 ↘ Resend API

7 SECURITY IMPLEMENTATION

Authentication & Authorization

- JWT HS256 tokens for stateless access and refresh flows
- Token type validation prevents misuse of access and refresh tokens
- Logout invalidates tokens via blacklist table
- bcrypt hashing secures passwords
- Role-based access enforced on endpoints with dependency checks
- New staff accounts require admin approval

Data Protection

- Secrets stored in environment variables, not source code
- CORS restricted to allowed frontend origins
- Pydantic schemas validate and sanitize all requests
- File uploads limited to 10MB and renamed safely
- HTTPS recommended in production

SLA & Audit Trail

- Every complaint status change is recorded in complaint_updates
- SLA breach creates audit entries with SLA_BREACH type
- Complaint ownership enforced by role and assignment

8 FUTURE SCOPE

Enhanced AI Capabilities

- Fine-tune prompts with historical complaint data
- Multi-turn guided complaint submission
- AI-generated resolution suggestions
- Duplicate complaint detection

Advanced Notification System

- Mobile push notifications via Web Push API
- SMS alerts for critical complaints
- User-configurable notification preferences
- Daily or weekly email digest summaries

Analytics & Reporting

- Interactive dashboards replacing static views
- Predictive analytics for complaint volume
- Campus heatmaps for location-based issues
- Export to Excel or CSV in addition to PDF

Mobile Application

- React Native app for students and staff
- Camera-based document uploads
- Biometric authentication support
- Offline complaint drafting and sync

Enterprise Features

- Multi-tenant support for multiple institutions
- LDAP/SSO integration for campus login systems
- Bulk complaint import via CSV
- Third-party API integrations for ERP or LMS
- GDPR and data retention policy support

9 CONCLUSION

CCMS provides a cohesive digital solution for campus complaint management. It combines AI classification, SLA enforcement, role-based access, real-time communication, and reporting in a scalable full-stack system.

The project demonstrates a mature architecture with React frontend, FastAPI backend, PostgreSQL persistence, secure JWT authentication, and integrated email notifications. Automation of complaint routing and escalation improves responsiveness and accountability while analytics support better decision making.

Key Achievements

- AI-powered complaint classification
- Real-time notification system
- Automated SLA enforcement
- Role-based access control
- Comprehensive audit trail
- In-app messaging
- Analytics dashboard
- PDF report export
- Rating and feedback system
- Secure authentication

10 RESULT

DRIVE LINK:

https://drive.google.com/drive/folders/11yo9NttBZ6OgnVR4iA5iY6_XELc7ZJPu?usp=drive_link

11 APPENDIX

11.1 Repository Information

Repository: CCMS

Owner: Shripad24

Branch: main

Link: <https://github.com/Shripad24/CCMS>

Structure:

/backend — FastAPI backend application

/frontend — React frontend application

README.md — Setup and usage guide

11.2 API Endpoints Summary

Method	Endpoint	Role	Description
--------	----------	------	-------------

POST	/api/v1/auth/register	Public	Register user
------	-----------------------	--------	---------------

POST	/api/v1/auth/login	Public	Login
------	--------------------	--------	-------

GET	/api/v1/auth/me	Authenticated	Get profile
-----	-----------------	---------------	-------------

POST	/api/v1/complaints/	Student	Submit complaint
------	---------------------	---------	------------------

GET	/api/v1/complaints/	Authenticated	List complaints
-----	---------------------	---------------	-----------------

GET	/api/v1/complaints/{id}	Authenticated	View complaint
-----	-------------------------	---------------	----------------

PATCH	/api/v1/complaints/{id}/status	Staff/Admin	Update status
-------	--------------------------------	-------------	---------------

POST	/api/v1/messages/	Authenticated	Send message
------	-------------------	---------------	--------------

GET	/api/v1/notifications/	Authenticated	Get notifications
-----	------------------------	---------------	-------------------

11.3 Database Schema Summary

users, departments, complaints, complaint_updates, messages, notifications, ratings, sla_policies, token_blacklist